PHD PROGRAM

Applied Data Science & Artificial Intelligence

UNIVERSITÀ DEGLI STUDI DI TRIESTE

Explainable Artificial Intelligence

Concept-based artificial intelligence



Outline

- Motivation of concept-based methods
- Concept-based methods overview
- Testing with Concept Activation Vectors (TCAV)
- Automatic Concept-based Explanations (ACE)

From Low-level to High-level features

Low-level Features are raw data representations such as pixels in images, waveforms in audio or single data points in tabular data. They are used directly by models but may not have a clear semantic (what is the semantic of single pixel?).

High-Level Features are instead more abstract and generally referable to human-recognizable concepts. For example, in an image of a dog, high-level features could include concepts like "fur texture," "four-legged structure," or "tail shape“.



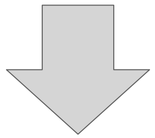
low-level features:
pixels, edges, …

high-level features:
fur texture,
four-legged structure,
tail shape, …

Motivation

Addressed issue:

- In several problems, computing importance scores for the input features is not informative enough (they may fail to capture high-level properties of the input data!)
- In these types of problems, the single feature does not provide a clear and unambiguous meaning from a semantic point of view
- Model's internal values (e.g., neural activations) can seem incomprehensible



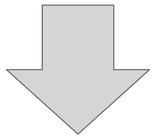
Concept-based methods for XAI try to address this issue by introducing "*concepts*", i.e., a way to define high-level properties for the input data, regardless of the available input features

A *concept* is essentially the definition of a user-friendly high-level feature that may be used to represent a possible dimension of the input data

ML state

What is the state of an ML model?

- Abstract definition
- We can try to give a mathematical intuition



- A possible state is defined *w.r.t.* a vector space E_m spanned by basis vectors e_m
- Basis vectors represent data such as input features and neural activations

Humans reason according to a different vector space

- As before, a possible state is defined *w.r.t.* a vector space E_h spanned by basis vectors e_h
- Basis vectors represent unknown human-interpretable concepts

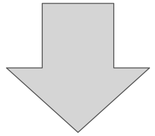
ML interpretation

Interpretation of an ML model:

A function $g : E_m \rightarrow E_h$ that maps states understandable by ML models to states understandable by humans

When the interpretability function is linear, we call it **linear interpretability**

Generally, the interpretability function is not perfect:
- it may fail to explain some aspects of the model input domain
- it will not cover all possible human-friendly concepts



High-level concepts are defined by using sets of example input data for the analyzed ML model

Doshi-Velez et al. “Towards a rigorous science of interpretable machine learning” (2017)

Concepts definition

How to define a concept?

A **concept** is defined by a set of observations (data) that captures its meaning. For instance, the concept “curly” can be defined by a set of hairstyles and texture images.

Main advantage: concepts are not constrained to input features or training data.

A concept can be defined by using an arbitrary new set of observations provided by the user.

Leveraging user-provided examples has been shown to be effective to enhance the interfacing experience between an ML model and a (expert or not) user.

Koh, P. W. et al. “Understanding black-box predictions via influence functions” (2017)
Kim, B. et al. “The Bayesian Case Model: A generative approach for case-based reasoning and prototype classification” (2014)
Klein, G. “Do decision biases explain too much” (1989)

Testing with CAV (TCAV)



Concept Activation Vector

A **Concept Activation Vector** (CAV) translates from E_m to E_h

It is a vector in the direction of the values (e.g., activations) of the set of examples (i.e., observations) of the concept at hand

Intuition:

Derive CAVs by training a **linear classifier** between *concept* examples and *random* counterexamples and then taking the vector orthogonal to the **decision boundary** (similar to local linearity approaches)

Testing with CAV (TCAV) is a *linear interpretability* method that uses directional derivatives to quantify the model prediction sensitivity to an underlying *high-level concept*, learned by a CAV

Been Kim et al. “Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV)” (2018)
Alain, G. et al. “Understanding intermediate layers using linear classifier probes” (2016)
Raghu, M. et al. “Singular vector canonical correlation analysis for deep understanding and improvement” (2017)
Bau, D. et al. “Network dissection: Quantifying interpretability of deep visual representations” (2017)
Szegedy, C. et al. “Intriguing properties of neural networks” (2013)

Testing with Concept Activation Vector

Main goals:

- ***Accessibility***: requires little to no ML expertise of user
- ***Customization***: adapts to any concept (e.g., gender) and is not limited to concepts considered during training
- ***Plug-in readiness***: works without any retraining or modification of the ML model
- ***Global quantification***: can interpret entire classes or sets of examples with a single quantitative measure, and not just explain individual data inputs

For instance, given an ML image model recognizing zebras, and a new, user-defined set of examples defining “striped”, TCAV can quantify the influence of striped concept to the “zebra” prediction as a single number.

Been Kim et al. “Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV)” (2018)

Testing with Concept Activation Vector

Scope of explanation:

TCAV perturbs data points towards a human-related concept to generate explanations

A generated explanation **holds for a specific class** rather than a specific data point

Specificity:

Mapping **latent dimensions** to human concepts

Hypothesis testing on any *concept* on the fly **that makes sense to the user for a trained network**

Been Kim et al. “Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV)” (2018)

Testing with Concept Activation Vector

Consider a neural network model with n features (i.e., variables)
A generic data observation (i.e., a data point) is represented by a vector $x \in \mathbb{R}^n$

Given a feedforward layer l with m neurons, the corresponding activation vector is represented by the function $f_l : \mathbb{R}^n \rightarrow \mathbb{R}^m$

First step:

Define a ***concept*** of interest ➔
- set of examples representing the concept
- independent dataset with the concept labeled

No constraints to pre-existing features, labels, or training data of the analyzed model and flexibility for even non-expert ML model analysts

Been Kim et al. “Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV)” (2018)

Testing with Concept Activation Vector

Second step:

Discover a vector in the space of activations at layer l that represents the *concept*

1. Apply the neural network to each observation representing the *concept*
2. Store the model activations at layer l
3. Perform the same operation for random observations with no *concept*
4. Define a CAV as the normal to a hyperplane separating random observations without a *concept* and observations with a *concept* in the space of model activations at layer l

Example: *concept* C "striped textures", we gather a set P_C of images of striped objects (positive set) and a set N of random images (negative set) ➔

- Train a binary linear classifier to discriminate between $\{f_l(x) : x \in P_C\}$ and $\{f_l(x) : x \in N\}$
- $v_C^l \in \mathbb{R}^m$ is the linear CAV ➔ encodes the direction of C in the space of activations at layer l

Been Kim et al. "Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV)" (2018)

Saliency Maps

Images dataset scenario:

Each image has a label ➔ classification task

- A CNN predicts the class of a given image by analyzing the generated logits
- By applying *SoftMax* to the generated logits, you obtain a probability distribution over the classes for that given image

Saliency maps use the gradients of logit values w.r.t. pixels ➔ $\frac{\partial h_k(x)}{\partial x_{a,b}}$

- $h_k(x)$ is the logit for data observation x and class k
- $x_{a,b}$ is the pixel at position (a, b) in x

This method computes through gradients the sensitivity of the output class k to changes in the magnitude of pixel (a, b)

Testing with Concept Activation Vector

In TCAV, the sensitivity of the prediction to changes in the input is measured towards the direction of a *concept* at activation layer l

Third step:

Compute the *conceptual sensitivity* of class k to concept C as the **directional derivative** $S_{C,k,l}(x)$

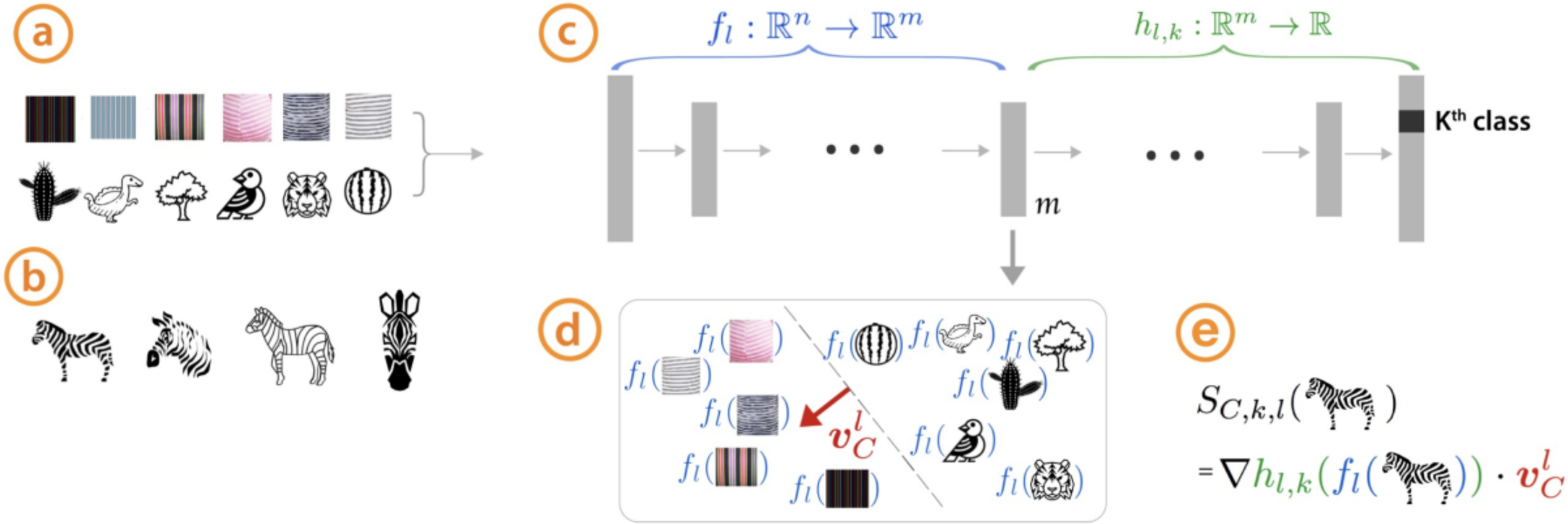
$$
\begin{aligned}
S_{C,k,l}(\boldsymbol{x}) &= \lim_{\epsilon \to 0} \frac{h_{l,k}(f_l(\boldsymbol{x}) + \epsilon \boldsymbol{v}_C^l) - h_{l,k}(f_l(\boldsymbol{x}))}{\epsilon} \\
&= \nabla h_{l,k}(f_l(\boldsymbol{x})) \cdot \boldsymbol{v}_C^l, \qquad (1)
\end{aligned}
$$

- $h_{l,k} : \mathbb{R}^m \rightarrow \mathbb{R}$ computes the logit for class k starting from activation layer l
  ($h_k(x)$ is a particular case of this when the activation layer matches the input layer)

Been Kim et al. “Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV)” (2018)

Testing with Concept Activation Vector

Thanks to the *conceptual sensitivity*, we can estimate a per-concept scalar quantity, computed on sets of observations, that measures the sensitivity of model predictions *w.r.t.* a given *concept*



Been Kim et al. “Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV)” (2018)

Testing with Concept Activation Vector

Fourth step:

Let k be a class label for our supervised learning problem
Let X_k be the set of all observations labeled with k

Compute the **TCAV score** (TCAV_Q) as follows

$$\text{TCAV}_{Q_{C,k,l}} = \frac{|\{\boldsymbol{x} \in X_k : S_{C,k,l}(\boldsymbol{x}) > 0\}|}{|X_k|} \qquad (2)$$

The **TCAV score** is the percentage of observations with class k whose activation layer l was positively influenced by concept C

Been Kim et al. “Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV)” (2018)

Testing with Concept Activation Vector

The **TCAV score** only depends on the sign of the *conceptual sensitivity*

Alternative **TCAV scores** can be defined by accounting for the magnitude of the *conceptual sensitivity*

The **TCAV score** allows *conceptual sensitivities* to be easily interpreted, globally for all observations of a given label

Been Kim et al. “Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV)” (2018)

Testing with Concept Activation Vector

Potential issue:

Learning non-meaningful CAVs ➔ for a given concept C, perform multiple training runs (e.g., 500) of the CAV with different samples of random observations (i.e., different N sets)

A *meaningful* concept should lead to **TCAV scores** that behave *consistently* across training runs

Suppose we fix the class k and the activation layer l, we train and compute multiple CAVs, each of them leads to a specific **TCAV score**

Null hypothesis: $\text{TCAV}_Q = 0.5$

Statistical test: two-sample t-test ➔ if null hypothesis is rejected, then the concept C is related to the class prediction in a *significant* way

Been Kim et al. “Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV)” (2018)

Testing with Concept Activation Vector

What to do if I want to relatively compare semantically related concepts?

For instance, “brown hair” and “black hair” are *semantically related concepts* ➔
– the corresponding CAVs are unlikely to be orthogonal
– relative comparisons are a good interpretative tool

Let C and D be two different *concepts* ➔ collect a set of inputs for each of them (P_C and P_D)
Train a **classifier** between $\{f_l(x) : x \in P_C\}$ and $\{f_l(x) : x \in P_D\}$ ➔ $v^l_{C,D} \in \mathbb{R}^m$ is the **CAV**
For a generic $f_l(x)$, what is the most relevant *concept* between C and D?

Intuitively, the **hyperplane** determined by $v^l_{C,D}$ yields to a $(m-1)$-dimensional *subspace* of the *vector space* $< \mathbb{R}^m, +, \bullet >$ over $\mathbb{R}$-field ➔ the **projection** of $f_l(x)$ onto this *subspace* measures whether x is more relevant to concept C or D

Been Kim et al. “Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV)” (2018)
Kim, B. et al. “Mind the gap: A generative approach to interpretable feature selection and extraction” (2015)
Doshi-Velez, F. et al. ” Graph-sparse lda: A topic model with structured sparsity” (2015)
Tibshirani, R. “Regression shrinkage and selection via the lasso” (1994)
Salvatore, C. et al. “Machine learning on brain mri data for differential diagnosis of parkinson’s disease and progressive supranuclear palsy” (2014)

Testing with Concept Activation Vector

Example:

Three *concepts*:
– “dotted”
– “striped”
– “meshed”

Likely to exist as internal representations, correlated or overlapping

Given P_{dot}, P_{stripe}, P_{mesh}, derive a relative **CAV** by building, for each, a negative input set by complement

For instance, for P_{stripe} as positive set, the negative set is $\{P_{dot} \cup P_{mesh}\}$

Been Kim et al. “Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV)” (2018)

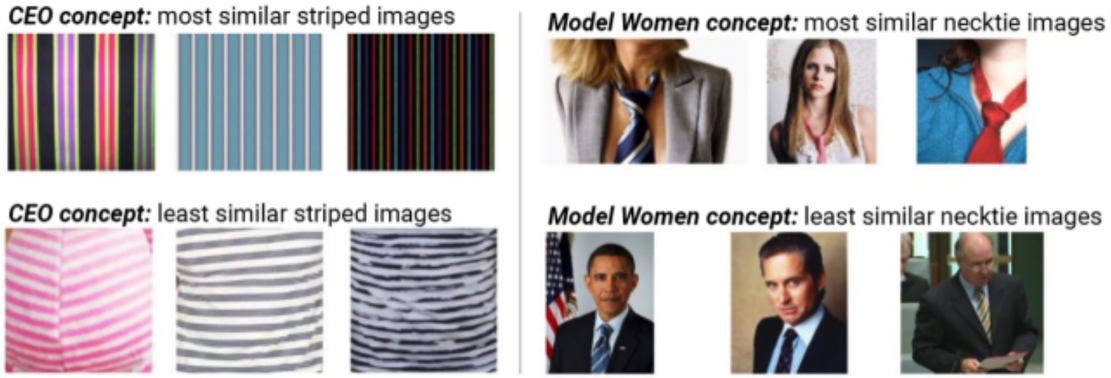
Results

Sorting images with CAVs:

We can use CAVs to sort images *w.r.t.* their relation to a *concept* C of interest:

- Consider an image $x^i \in \mathbb{R}^n$ and its l-layer embedding $f_l(x^i) \in \mathbb{R}^m$
- Consider a CAV $v_C^l \in \mathbb{R}^m$ for *concept* C
- Consider the cosine similarity between v_C^l and $f_l(x^i)$

Sort a set of images (not in the training set) by cosine similarity computed with the **CAV**



Been Kim et al. “Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV)” (2018)

Results

A controlled experiment with ground truth:

Demonstrate that **TCAV** can interpret the function learned by a neural network in a controlled setting where ground truth is known

Dataset: set of images with three classes (zebra, cab, cucumber) and potentially noisy captions written in the image (a noise parameter $p \in [0, 1]$ determines the probability that the caption agrees with the class)



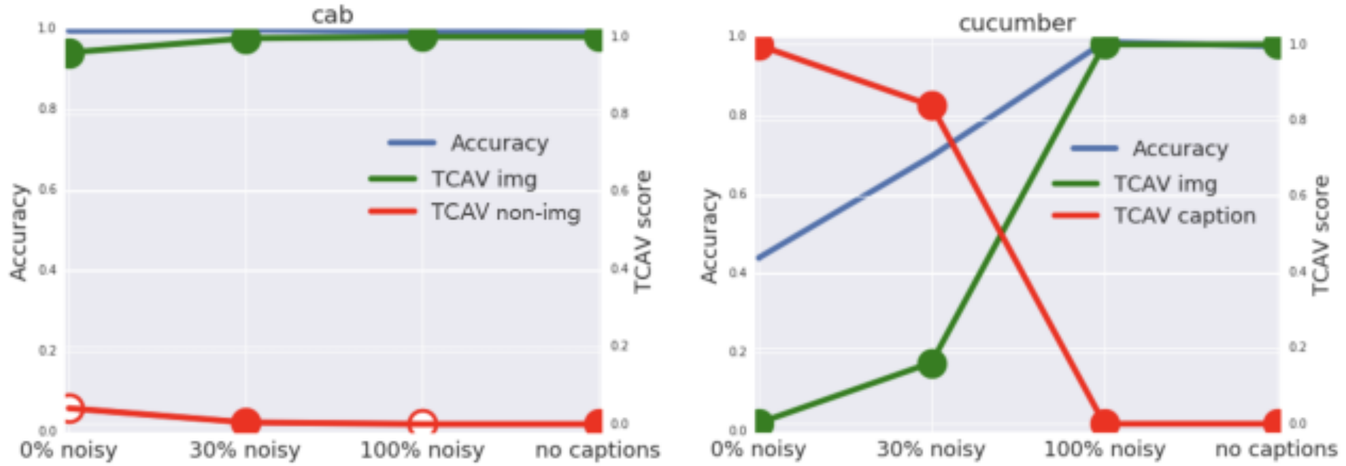
cab image | cab image with caption | cucumber image | cucumber with caption

Four networks are trained with different p, and network is tested on no-caption images to obtain an approximated ground truth for which *concept* the network paid attention ➔ if the network used the image concept for classification, the performance should remain high

Been Kim et al. “Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV)” (2018)

Results

Two sets of **CAVs**:
- image CAVs using each class no-caption images
- caption CAVs using caption images where pixels of the real image are randomly shuffled



TCAV score closely mirrors the *concept* that the network paid attention to:
For cabs, the network used the image *concept* more than the caption *concept* regardless of p
For cucumber, the network sometimes used the image *concept* and sometimes the caption *concept*

Been Kim et al. “Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV)” (2018)

Outcomes

TCAV is an attempt to create a user-friendly linear interpretation of the internal state of a neural network

Questions about model decisions may be answered in terms of high-level *concepts*

Concepts do not need to be known at training time

Possible future directions:

TCAV for other types of data ➔ text, audio, video, sequences...

Automatic *concept* identification

Been Kim et al. “Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV)” (2018)

Automatic Concept-based Explanations (ACE)



Automatic Concept-based Explanations

Possible issues of TCAV:

- *Concepts* may be difficult to define
- Once a *concept* is defined, examples collection is subject to human bias
- Humans may fail in defining the right *concepts*
- The space of all possible *concepts* is potentially unlimited (or unclear)
- *What hidden layer l should we choose?*

Need an automatic procedure for visual *concepts* identification

Automatic Concept-based Explanations (ACE) is a *framework* that automatically identifies higher-level *concepts* which are meaningful to humans and are important for the ML model.

It aggregates related local image segments across diverse data.

Been Kim et al. “Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV)” (2018)
Amirata Ghorboni et al. “Towards Automatic Concept-based Explanations” (2019)

Automatic Concept-based Explanations

Desired properties for concept-based explanations of a ML model:

- ***Meaningfulness***: a *concept* example is semantically meaningful on its own (e.g., a group of pixels containing a texture *concept* or an object part *concept*)
- ***Coherency***: examples of a *concept* should be perceptually similar to each other while being different from examples of other *concepts*
- ***Importance***: a *concept* is important for the prediction of a class if its presence is necessary for the true prediction of samples in that class

ACE is a global explanation method that explains an entire class in a trained classifier without the need for human supervision

Amirata Ghorboni et al. “Towards Automatic Concept-based Explanations” (2019)

Automatic Concept-based Explanations

Setting:

ACE takes two inputs: a **trained classifier** and a **set of images of a given *class***

The main output consists in a set of *concepts* that are present in the given *class*
Importance of each *concept* is computed and returned

Group of pixels ➔ segment (in image data, *concepts* are present as segments)

We want to answer the following question:
how to extract all *concepts* of a given *class*?

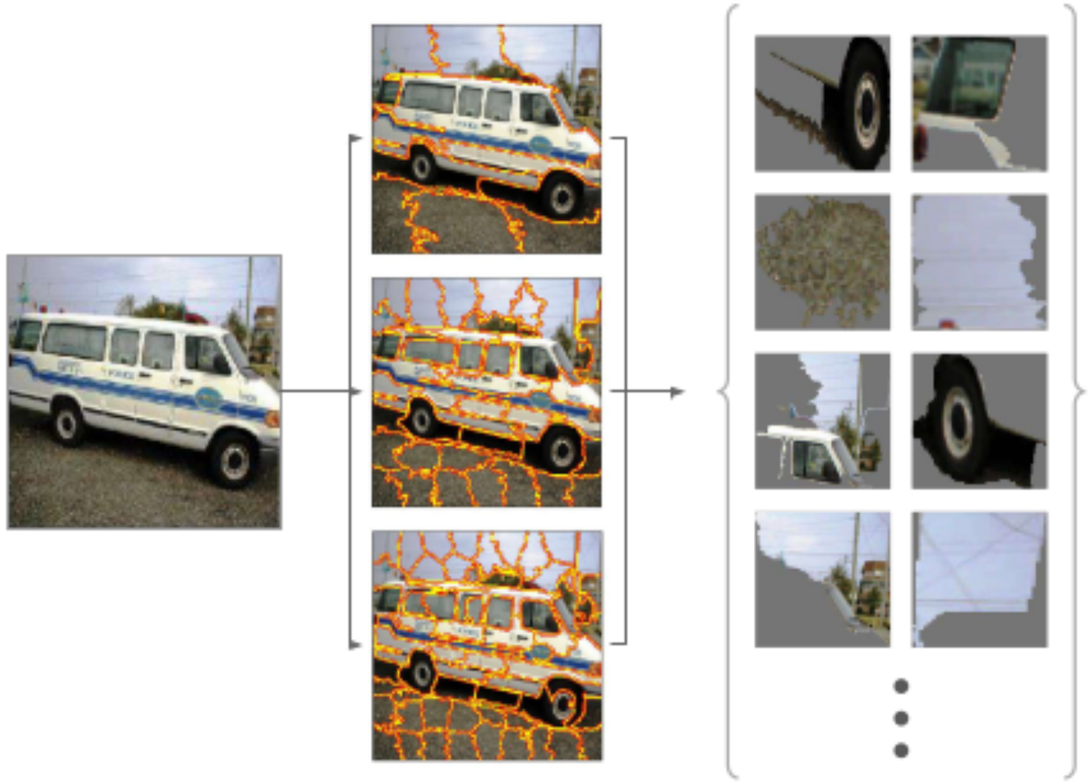
Amirata Ghorboni et al. “Towards Automatic Concept-based Explanations” (2019)

Automatic Concept-based Explanations

First step:

For each *class* image:

1. perform segmentation with multiple (e.g., 3) resolutions
2. store the extracted segments
3. **pool of segments for the given *class***



Amirata Ghorboni et al. “Towards Automatic Concept-based Explanations” (2019)

Automatic Concept-based Explanations

Second step:

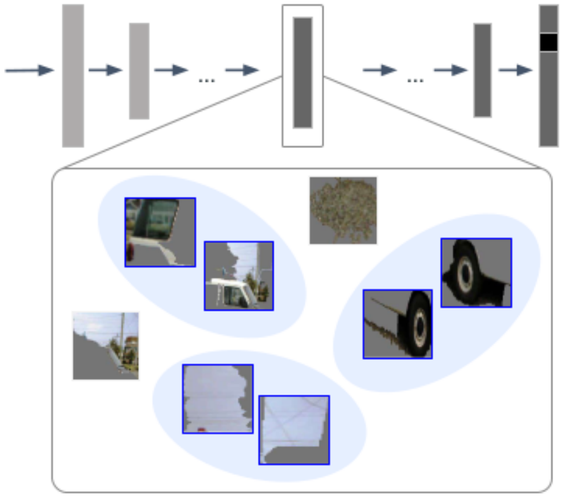
Perform a clustering of segments

Previous works highlighted that **euclidean distance** in the activation space of final layers is a good perceptual similarity metric as regards state-of-the-art CNNs

Segments are **resized** to match the input layer size of the CNN

The pre-trained CNN is applied to each segment and an embedding is stored

Similar segment embeddings are clustered together



Amirata Ghorboni et al. “Towards Automatic Concept-based Explanations” (2019)
P. Dabkowski et al. “Real time image saliency for black box classifiers” (2017)
O. Russakovsky et al. ”Imagenet large scale visual recognition challenge” (2015)
R. Zhang et al. “The unreasonable effectiveness of deep features as a perceptual metric” (2018)

Automatic Concept-based Explanations

For each cluster, outlier segments are removed

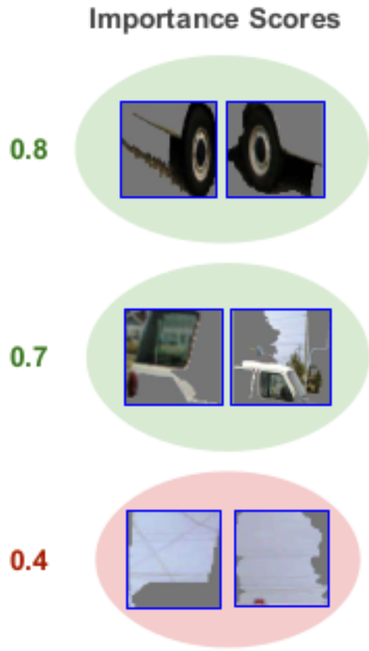
Each cluster of segments is a *concept*

Third step:

Now, we have a *concept* for each cluster ➔ How many of them are useful *concepts*?

For each *concept*, given the *class* at hand and the layer l corresponding to the embedding space exploited during the clustering procedure, **compute its TCAV score by using the segments in the cluster as *concept* examples**

By fixing a threshold, we can extract the most important *concepts*



Amirata Ghorboni et al. “Towards Automatic Concept-based Explanations” (2019)
Been Kim et al. “Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV)” (2018)

Automatic Concept-based Explanations

How ACE can achieve the three desiderata?

In the **first step**, **ACE** leverages simple and fast super-pixel segmentation methods (widely used in hierarchical segmentation literature) ➔ any desired level of resolution but lower segmentation quality

In the **second step**, **ACE** leverages an ImageNet-trained CNN to compute segment embeddings

In the **third step**, **ACE** leverages the **TCAV** score to assign importance to each discovered *concept*

Satisfying the desiderata through ACE is limited to the performance of segmentation, clustering, outliers removal, and reliability of CNNs as a similarity metric

Amirata Ghorboni et al. “Towards Automatic Concept-based Explanations” (2019)
X. Wei et al. ” Superpixel hierarchy” (2018)
Been Kim et al. “Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV)” (2018)

Results

Interpretation of Inception-V3 model trained on ILSVRC2012 dataset (ImageNet):

Selected classes: 100 out of the 1000 available
Number of examples for each concept: ~ 10 or 20 (50 examples per class)
Segmentation step: SLIC with resolutions 15, 50, 80 segments for each image
Similarity metric: euclidean distance in the “mixed_8” layer of the ImageNet-trained ***Inception-V3*** model

Earlier layers are suited for textures and colors
Latter layers are suited for objects

Clustering algorithm: K-Means ➔ need to estimate the number of *concepts*
Outliers removal: euclidean distance to the cluster centers

Amirata Ghorboni et al. “Towards Automatic Concept-based Explanations” (2019)
O. Russakovsky ”Imagenet large scale visual recognition challenge” (2015)
C. Szegedy et al. “Rethinking the inception architecture for computer vision” (2016)
R. Achanta et al. “Slic superpixels compared to state-of-the-art superpixel methods” (2012)

Results

Output for three classes:

For each class, the top-4 *concepts* are shown (with three random examples for each *concept*)

Several concepts are highlighted:

- Lionfish spines
- Lionfish skin texture
- Car wheel
- Car window
- Police logo
- Basketball player arm
- Basketball player shirt



Amirata Ghorboni et al. “Towards Automatic Concept-based Explanations” (2019)

Results

Human experiments:

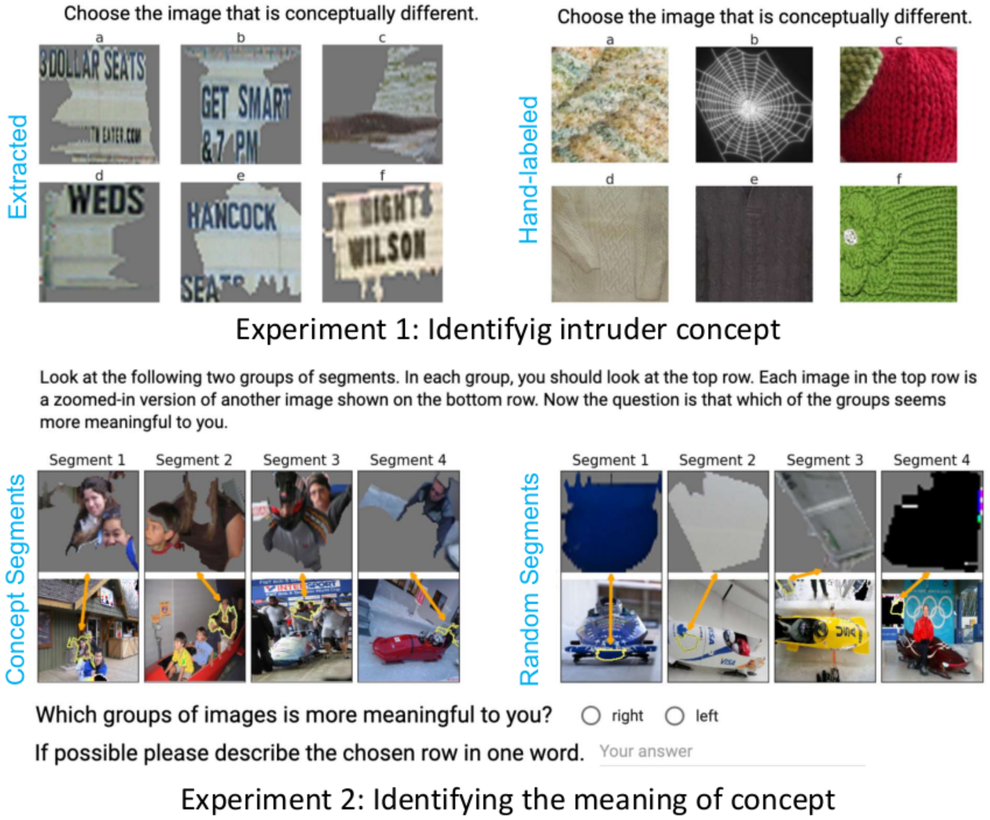
Intruder detection:

30 participants and 34 questions (4 for user training, 15 human-labeled *concepts*, 15 ACE *concepts*) ➔ human-labeled concepts were answered (14.6/15) (±0.7) correctly, ACE concepts (14.9/15) (±0.3)

Concepts meaningfulness:

30 participants and 15 questions

Testing meaningfulness ➔ correct option was chosen (14.3/15)(±1.0) ➔ *also, ACE concepts were described with same words or synonyms*



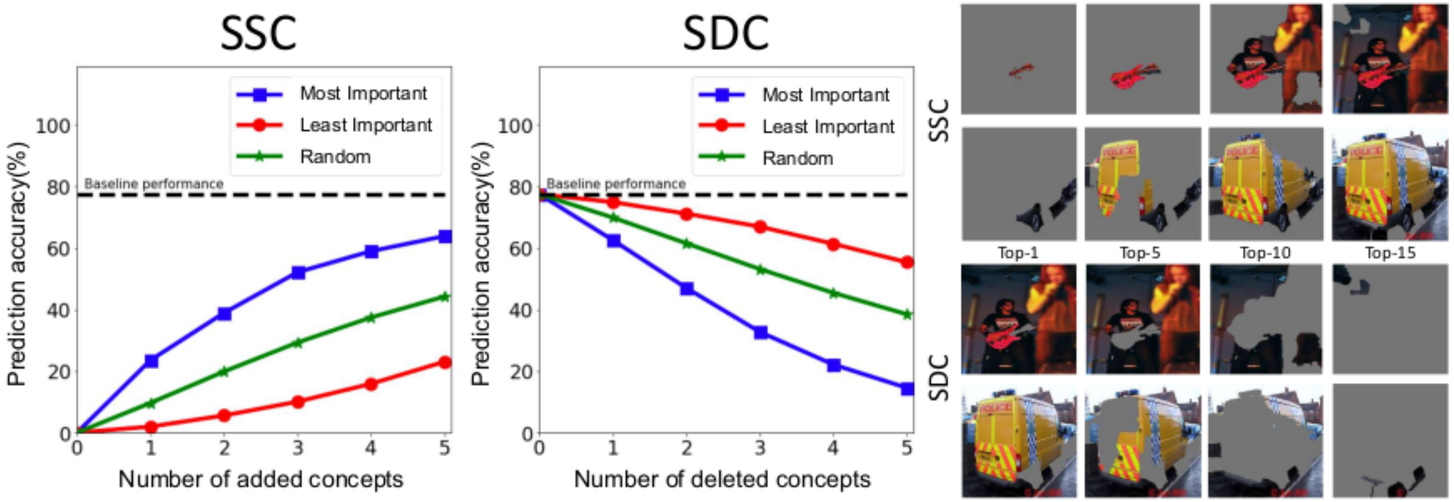
Amirata Ghorboni et al. “Towards Automatic Concept-based Explanations” (2019)

Results

Relevance of important concepts:

Smallest Sufficient Concepts (SSC) and **Smallest Destroying Concepts** (SDC) for 1000 randomly sampled images in the ImageNet validation set ➔ each image is segmented ➔ each segment is assigned to a previously discovered *concept* ➔ the assigned *concept* is the one that best fit the segment, i.e., the most similar one according to the similarity between the segment and the *concept* cluster

- 80% baseline accuracy if no segmentation is applied
- Top-5 concepts are enough to reach 80% of original accuracy
- Removing top-5 concepts results in large misclassifications



Amirata Ghorboni et al. “Towards Automatic Concept-based Explanations” (2019)
P. Dabkowski et al. “Real time image saliency for black box classifiers” (2017)

Outcomes

ACE is an attempt to automatically detect semantically meaningful *concepts* from image data by grouping similar image segments

This can be applied only to image data
Concepts are present only in segments ➔ more abstract *concepts* may be present

Possible future directions:

ACE for other types of data ➔ text, audio, video, sequences...
More sophisticated dictionary learning approaches on the representation space
Hyper-parameters tuning for each class separately

Amirata Ghorboni et al. “Towards Automatic Concept-based Explanations” (2019)

THIS IS THE END



Thanks for the attention!